

Smart Asset Management and Resource Allocation Platform

1. Problem Understanding

Organizations frequently manage shared resources such as cameras, audio systems, lighting equipment, event infrastructure, and recording devices. Traditional inventory management methods often rely on spreadsheets, manual registers, and informal communication channels, leading to poor visibility, resource conflicts, delayed returns, and inefficient utilization.

The objective of this project is to build a centralized web-based platform that enables efficient asset tracking, booking, allocation, approval workflows, maintenance management, and operational analytics.

The platform supports both administrators and users through role-based access control and provides complete visibility into asset availability, booking requests, issuance records, and inventory utilization.

2. System Architecture

Architecture Overview

Frontend Layer

↓

REST API Layer

↓

Backend Business Logic

↓

Authentication & Authorization

↓

MySQL Database

Frontend

- React.js
- Vite
- Material UI
- React Router

- Axios
- Chart.js

Responsibilities:

- User Interface
- Dashboard Visualizations
- Asset Browsing
- Booking Management
- QR Scanner Operations

Backend

- Node.js
- Express.js

Responsibilities:

- API Development
- Business Logic
- Authentication
- Authorization
- Asset Allocation
- Inventory Validation

Database

- MySQL

Responsibilities:

- Persistent Storage
 - Transaction Management
 - Data Integrity
 - Relationship Management
-

3. Functional Modules

User Authentication Module

Features:

- User Registration

- User Login
- JWT Authentication
- Password Encryption using Bcrypt
- Google OAuth Login
- Role-Based Authorization

Roles:

- Administrator
 - User
-

Inventory Management Module

Administrators can:

- Create Assets
- Update Asset Information
- Delete Assets
- Categorize Assets
- Manage Inventory Quantities

Asset Information:

- Asset Name
 - Category
 - Description
 - Quantity
 - Status
 - Condition
-

Booking Management Module

Users can:

- Browse Assets
- Search Inventory
- Submit Booking Requests
- Track Request Status

Booking States:

- Pending

- Approved
- Rejected
- Issued
- Returned

Inventory validation ensures requests never exceed available quantities.

Approval Workflow

Administrators review booking requests and perform:

- Approval
- Rejection
- Asset Allocation

This ensures accountability and controlled resource distribution.

Asset Issue and Return Module

Features:

- Asset Issuance
- Due Date Tracking
- Return Processing
- Overdue Monitoring

Inventory quantities are updated automatically throughout the lifecycle.

Notification Module

Provides notifications for:

- Booking Approvals
 - Booking Rejections
 - Asset Returns
 - Operational Alerts
-

Audit Log Module

Tracks:

- Asset Creation
- Asset Updates
- Booking Actions
- Approval Decisions
- Returns

This improves transparency and accountability.

Maintenance Management Module

Tracks:

- Asset Health
- Maintenance Records
- Service History
- Repair Status

Asset conditions:

- Excellent
 - Good
 - Damaged
-

QR-Based Asset Operations

Administrators can:

- Generate Asset QR Codes
 - Scan Assets
 - Retrieve Asset Information
 - Simplify Issue and Return Processes
-

Dashboard Analytics

Provides:

- Asset Utilization Statistics
- Active Bookings

- Inventory Overview
- Operational Insights
- Resource Allocation Trends

Visualization Components:

- Bar Charts
 - Pie Charts
 - Summary Cards
-

4. Database Design

Users

Stores authentication and role information.

Attributes:

- id
 - name
 - email
 - password_hash
 - role
 - created_at
-

Asset Categories

Groups assets into categories.

Attributes:

- id
 - category_name
 - description
-

Assets

Stores inventory information.

Attributes:

- id
 - asset_name
 - category_id
 - description
 - total_quantity
 - available_quantity
 - asset_status
 - asset_condition
-

Booking Requests

Stores booking records.

Attributes:

- id
 - user_id
 - asset_id
 - requested_quantity
 - start_date
 - end_date
 - purpose
 - status
-

Asset Issues

Tracks issued assets.

Attributes:

- id
 - booking_id
 - issue_date
 - due_date
 - return_date
 - issue_status
-

Notifications

Stores system notifications.

Attributes:

- id
 - user_id
 - title
 - message
 - is_read
-

Audit Logs

Stores user actions.

Attributes:

- id
 - user_id
 - action
 - details
-

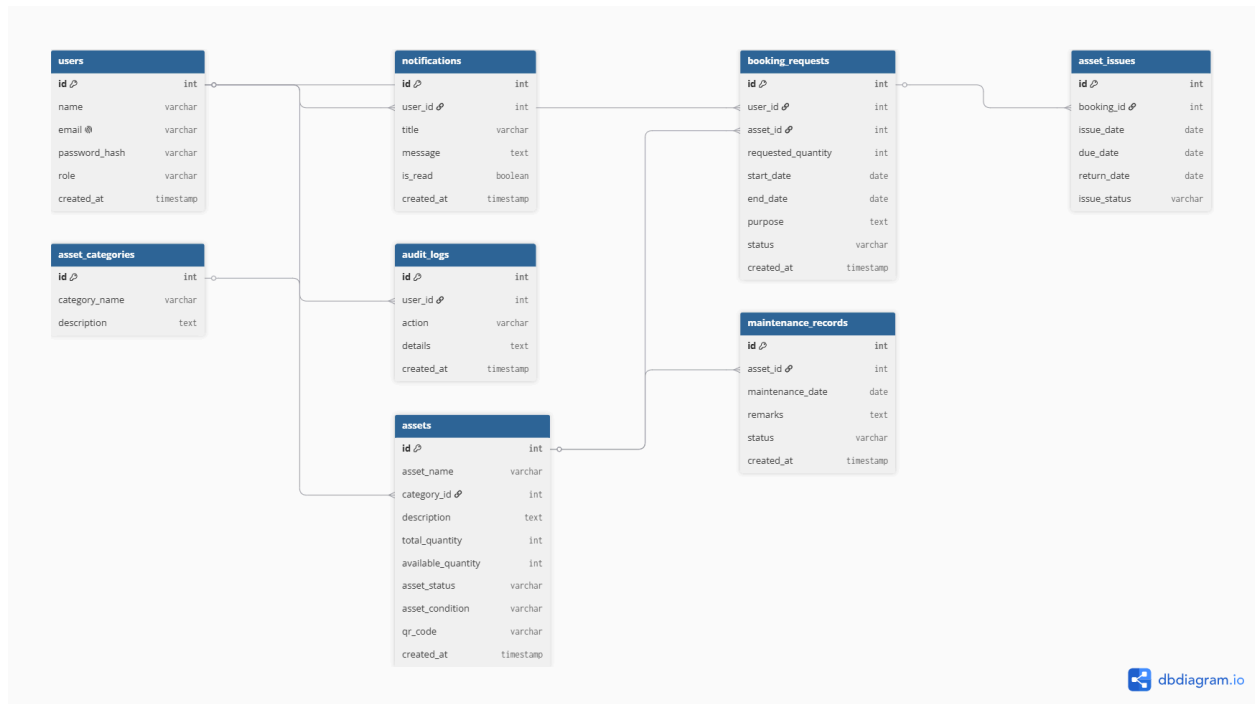
Maintenance Records

Stores maintenance history.

Attributes:

- id
 - asset_id
 - maintenance_date
 - remarks
 - status
-

5. Entity Relationship Diagram



6. API Overview

Authentication APIs

- POST /register
- POST /login
- GET /profile

Asset APIs

- GET /assets
- POST /assets
- PUT /assets/:id
- DELETE /assets/:id

Booking APIs

- POST /bookings
- GET /bookings
- PUT /bookings/:id/approve
- PUT /bookings/:id/reject

Issue APIs

- POST /issue
- POST /return

Dashboard APIs

- GET /dashboard

Notification APIs

- GET /notifications

Maintenance APIs

- POST /maintenance
- GET /maintenance

Audit APIs

- GET /auditlogs
-

7. Security Measures

Implemented Security Features:

- JWT Authentication
 - Password Hashing using Bcrypt
 - Protected Routes
 - Role-Based Authorization
 - Input Validation
 - Secure API Access
-

8. Design Decisions

Key design choices:

1. JWT authentication for stateless session management.
2. MySQL selected for structured relational data.
3. Role-based access control for administrative security.
4. Audit logs added for accountability.
5. QR code integration improves operational efficiency.
6. Maintenance records extend asset lifecycle management.

7. Modular MVC architecture improves maintainability.
-

9. Scalability Considerations

The platform is designed to support:

- Thousands of assets
- Multiple departments
- Concurrent bookings
- Large user bases
- Additional notification channels
- Cloud deployment

Future improvements include:

- Docker deployment
- Email notifications
- Advanced analytics
- Mobile application support
- Predictive maintenance